SystemVerilog中的生命周期lifetime

1.SystemVerilog中的生命周期lifetime

- 在SystemVerilog中lifetime主要是针对变量而言的，主要用于表述变量在整个仿真过程中是否一直占用空间。在SystemVerilog中，lifetime主要有两种：一种是static静态的，一种是automatic动态的，一般静态变量存在于整个仿真过程中直到仿真结束，而动态则仅存在于其被调用的过程中。

2.声明于module/program/interface/checker/task/function之外的变量，具有静态的生命周期

- **y显示的声明为静态变量，x为隐含静态变量**，即x、y具有静态生命周期。仿真结果中，**x和y属于全局空间$unit中**。其实x、y在示例中的声明特点是x和y声明于module之外，此时不管是不是将x和y声明为static，x和y都具有静态的生命周期，在模块top_tb中，x和y对于其中的p1和p2都是可观测可访问的。

```
`timescale 1 ns / 1 ps
       int x = 8;
static int y = 9;
module top_tb;

initial begin // p1
        $display("@%0t : in p1 : 1st : x = %0h",$time,x);
        $display("@%0t : in p1 : 1st : x = %0h",$time,y);
    #10 $display("@%0t : in p1 : 2nd : x = %0h",$time,x);
        $display("@%0t : in p1 : 2nd : x = %0h",$time,y);
end

initial begin // p2
    #5 x = 3;y = 4;
end
endmodule
```



```
@0 : in p1 : 1st : x = 8
@0 : in p1 : 1st : x = 9
@10000 : in p1 : 2nd : x = 3
@10000 : in p1 : 2nd : x = 4
Simulation complete, time is 10000
```

- 位于**module/program/interface/checker/task/function**之外的变量是不能声明为**automatic**的。

3.声明于module/program/interface/checker中，但是位于task/function/process之外的变量是module/program/interface/checker的局部变量，具有静态生命周期

- x和y都在module中，但没有位于任何方法中，此时x和y的生命周期与模块top_tb一样，都是静态的，**p2对于x和y的修改在p1中是可以观测到的**，此时x和y从仿真开始就一直存在，所以此时的**x和y是不能显式的声明为automatic的**，否则将会导致仿真编译错误

```
`timescale 1 ns / 1 ps
module top_tb;
          int x = 8;
static    int y = 9;

initial begin // p1
        $display("@%0t : in p1 : 1st : x = %0h",$time,x);
        $display("@%0t : in p1 : 1st : x = %0h",$time,y);
    #10 $display("@%0t : in p1 : 2nd : x = %0h",$time,x);
        $display("@%0t : in p1 : 2nd : x = %0h",$time,y);
end

initial begin // p2
    #5 x = 3;y = 4;
end
endmodule
```

硅芯思见

```
@0 : in p1 : 1st : x = 8
@0 : in p1 : 1st : x = 9
@10000 : in p1 : 2nd : x = 3
@10000 : in p1 : 2nd : x = 4
Simulation complete, time is 10000
```

4.声明于“静态方法”和块中的变量可以显式的指定为automatic，这时这些变量生命周期仅存在于方法或者过程块的每次调用中，并且在每次调用时，这些变量都会被重新初始化

- 仅temp的生命周期是automatic动态的，其他变量仍未static，即具有静态的生命周期。

```
`timescale 1 ns / 1 ps
module top_tb;
parameter DLY = 2;
reg [1:0] result_1,result_2;
reg       co_1,co_2;
initial fork
    begin // p1
        #1 add2x2(result_1,co_1);
    end
    begin // p2
        #(DLY+1) add2x2(result_2,co_2);
        #(DLY*4);
    end
join

// default static lifetime
task add2x2(output [1:0] result,output co);
automatic reg [1:0] temp = 2'b01;
begin
    #1 $display("@%0t : STATIC TASK : TEMP IS %0d",$time,temp);
    #(DLY*2) {co,result} = temp + 2'b10;
    temp = 2'b10;
end
endtask
endmodule
```

硅芯思见

- 可以通过对任务直接进行静态或者动态声明，从而可以影响方法中所有变量的属性。

```
`timescale 1 ns / 1 ps
module top_tb;
parameter DLY = 2;
reg [1:0] result_1,result_2;
reg       co_1,co_2;
initial fork
    begin // p1
        #1 add2x2(result_1,co_1);
    end
    begin // p2
        #(DLY+1) add2x2(result_2,co_2);
        #(DLY*4);
    end
join


task automatic add2x2(output [1:0] result,output co);
reg [1:0] temp = 2'b01;
begin
    #1 $display("@%0t : AUTOMATIC TASK : TEMP IS %0d",$time,temp);
    #(DLY*2) {co,result} = temp + 2'b10;
    temp = 2'b10;
end
endtask
endmodule
```

硅芯思见

- 如果将其中“动态”方法中的变量声明staitc，那么这些变量将具有静态的生命周期。



- 方法被声明为了automatic，其中所有变量包括参数默认为static，但是因为其中的变量temp被声明为了static，temp此时具有静态的生命周期，所以从仿真结果我们也可以看到，方法的第二次调用受到了第一次调用时最后对temp进行的赋值修改。
- 对方法进行声明周期描述时，**automatic和static关键字位置是在task/function和方法名之间**

5.通过对module/interface/package或者program指定生命周期修饰符，从而影响定义在这些结构中的方法或者块中变量的生命周期属性

- add2x2中所有的变量和参数生命周期受到program设置的automatic的影响。

```
`timescale 1 ns / 1 ps
program automatic top_tb;
parameter DLY = 2;
reg [1:0] result_1,result_2;
reg       co_1,co_2;

initial fork
    begin // p1
        #1 add2x2(result_1,co_1);
    end
    begin // p2
        #(DLY+1) add2x2(result_2,co_2);
        #(DLY*4);
    end
join

task add2x2(output [1:0] result,output co);
reg [1:0] temp = 2'b01;
begin
    #1 $display("@%0t : AUTOMATIC TASK : TEMP IS %0d",$time,temp);
    #(DLY*2) {co,result} = temp + 2'b10;
    temp = 2'b10;
end
endtask
endprogram
```

硅芯思见

- 但是需要注意，对module/interface/package或者program指定生命周期修饰符(automatic或者static)，**只影响定义在这些结构中的方法或者块中变量的生命周期属性，并不影响定义在这些结构中的变量**

- 虽然将program声明为具有动态生命周期，x在两个initial块中分别进行了赋值访问操作，但通过仿真结果我们也可以看到这两个过程块对于x的修改都影响到了另一个过程块中的x的访问。

```
`timescale 1 ns / 1 ps
program automatic top_tb;
int x = 3; // default : static var

initial begin
    $display("@%0t : 1st : x is %0h",$time,x);
    #10;
    $display("@%0t : 2nd : x is %0h",$time,x);
    #2 x = 5;
end
initial begin
    #5 x = 4;
    #8 $display("@%0t : 3rd : x is %0h",$time,x);
end
endprogram
```

硅芯思见

5.总结

- 声明于module/program/interface/checker/task/function之外的变量，具有静态生命周期；
- 位于module/program/interface/checker/task/function之外的变量是不能通过automatic进行修饰声明的；
- 声明于module/program/interface/checker中，但是位于task/function之外，则变量是module/program/interface/checker中的局部变量，具有静态生命周期，并且不能声明为automatic；
- 声明于静态方法和块中的变量可以显式的指定为automatic，这时这些变量生命周期仅存在于方法或者块的每次调用中，并且在每次调用时，这些变量都会重新被初始化；
- 对方法直接进行静态或者动态生命周期声明，从而可以影响方法中所有变量的生命周期属性；
- 对module/program/interface/checker中的方法直接进行动态或者静态生命周期的声明，可以影响方法中变量的生命周期的属性；
- 对module/program/interface/checker指定automatic或者static从而影响定义在这些结构中的方法或者块中的变量的生命周期；
- 具有静态生命周期的方法实际上作用对象是方法中的变量或者参数；
- *静态变量表示用户期望在仿真开始时对变量仅进行一次初始化*；
- 变量如果没有在过程块或者方法中，是不能使用automatic进行声明；
- 静态变量声明的时候进行的初始化操作将发生在所有initial或者always语句启动之前；